TECHSTORE

DOCUMENTO DE COBERTURA DE TESTING

Evaluación Parcial 2 - DSY1104 Desarrollo Full Stack II

Equipo de desarrollo

Rosa Medina · Thiara Rojas · Andrea Huenumilla · Luna Bustamante

Versión grupal final · 20 pruebas ejecutadas

React · Bootstrap · Jasmine · Karma · ChromeHeadless · Istanbul

19 de octubre de 2026

Índice

1. Propósito y criterio del documento....3
2. Herramientas y configuración del entorno de pruebas....3
3. Mocks, spies y MemoryRouter....4
4. Diez pruebas base de referencia....5
5. Por qué la suite final tiene 20 pruebas....5
6. Ejecución final: pruebas y build....6
7. Cobertura global....7
8. Evidencias de módulos principales....8
9. Persistencia, datos y contexto....9
10. Páginas y validaciones....9
11. Componentes pequeños y Navbar....10
12. Análisis final y alcance de la medición....11
13. Conclusiones....11
14. Referencias....12

1. Propósito y criterio del documento

Este documento presenta de forma grupal la estrategia de testing aplicada a TechStore. Se documentan las herramientas y la configuración del entorno de pruebas, el uso de mocks y spies, las 10 pruebas base utilizadas como referencia para cumplir la pauta, la optimización realizada durante la integración final, que llevó la suite a 20 pruebas ejecutadas correctamente, y la verificación del build de producción.

El documento no asigna nombres individuales a cada prueba: la evidencia se organiza por módulo y por objetivo de prueba.

2. Herramientas y configuración del entorno de pruebas

Las pruebas se ejecutan con el comando npm test, definido en package.json como karma start karma.conf.cjs. Todo el entorno se configura en ese archivo, y las dependencias de testing están declaradas como devDependencies del proyecto.

Herramienta	Versión	Función en TechStore
Jasmine (jasmine-core, karma-jasmine)	7.0.2 / 5.1.0	Define describe, it, expect, matchers, createSpy y spyOn.
Karma	6.4.4	Ejecuta la suite de pruebas y entrega el resultado.
ChromeHeadless (karma-chrome-launcher)	3.2.0	Chrome sin interfaz, utilizado como navegador de pruebas.
Webpack y karma-webpack	5.111.1 / 5.0.1	Empaquetan cada archivo .spec.jsx con sus dependencias.
Babel (core, preset-env, preset-react) y babel-loader	8.0.x / 10.1.1	Transpilan JSX y JavaScript moderno.
Istanbul (babel-plugin-istanbul) y karma-coverage	8.0.0 / 2.2.1	Instrumentan el código y generan el reporte de cobertura.
karma-spec-reporter	0.0.37	Muestra en la terminal el nombre y el resultado de cada prueba.
style-loader y css-loader	4.0.0 / 7.1.5	Permiten que los componentes importen CSS durante las pruebas.

2.1 Qué configura karma.conf.cjs

Opción	Valor en TechStore
frameworks	jasmine y webpack.
files y preprocessors	Detecta todos los archivos src/**/*.spec.jsx y los procesa con Webpack.
babel-loader	preset-env (objetivo Chrome 100) y preset-react con runtime automático.
Plugin istanbul	Instrumenta el código para medir cobertura y excluye los archivos *.spec.jsx y la carpeta src/test.
reporters	spec (resultado por prueba) y coverage (reporte de cobertura).
coverageReporter	Genera un reporte html y un resumen de texto en la carpeta coverage.
browsers y singleRun	ChromeHeadless; la suite se ejecuta una vez y termina.

2.2 Ayudante montar.jsx

El archivo src/test/montar.jsx crea un contenedor temporal, renderiza cada componente dentro de un MemoryRouter usando act() y devuelve el contenedor junto con una función desmontar() para limpiar.

Incluye además escribir(), que simula la escritura en un campo, y boton(), que localiza un botón por su texto. Esto permite aislar el comportamiento bajo prueba sin levantar toda la aplicación.

3. Mocks, spies y MemoryRouter

Técnica	Qué hace	Dónde se usa
jasmine.createSpy()	Crea una función falsa que registra si fue llamada y con qué argumentos.	ProductCard (onAgregar), QuantitySelector (onAumentar y onDisminuir) y Navbar (onCerrarSesion).
spyOn(Storage.prototype, 'getItem') y 'setItem'	Reemplaza temporalmente la lectura y la escritura de localStorage para controlar lo que devuelve y verificar lo que se guarda.	Carrito + localStorage, y las pruebas de Registro, Login y sesión.
spyOn(Storage.prototype, 'removeItem')	Verifica el borrado de datos, por ejemplo al cerrar la sesión.	Prueba de Login y sesión.
MemoryRouter	Router de React Router que funciona en memoria, sin abrir rutas reales del navegador.	montar.jsx lo aplica a todas las pruebas; Home lo usa para comprobar la navegación hacia Productos.
act()	Agrupa las interacciones para que React termine de actualizarse antes de comprobar el resultado.	Pruebas con clics, escritura o cambios de estado.

Ejemplos tomados de las pruebas del proyecto:

```
const cerrar = jasmine.createSpy('onCerrarSesion')      // Navbar.spec.jsx
expect(cerrar).toHaveBeenCalledTimes(1)
spyOn(Storage.prototype, 'getItem').and.callFake(...)   // validacionesThiara.spec.jsx
```

> **Concepto clave**
> Un spy observa o reemplaza temporalmente una función para controlar el escenario de prueba y verificar interacciones sin ejecutar dependencias externas reales. En TechStore, el mock de localStorage se implementa con spyOn sobre Storage.prototype.

4. Diez pruebas base de referencia

N°	Módulo	Qué verifica	Tipo	Resultado
1	ProductCard	Renderiza nombre, precio e imagen recibidos por props.	Render / props	Exitosa
2	ProductCard	El botón Agregar ejecuta el callback con el producto.	Evento / spy	Exitosa
3	Registro	Valida los datos del formulario y el flujo de creación de usuario.	Validación / estado	Exitosa
4	Login	Valida credenciales y persistencia de la sesión.	Validación / storage	Exitosa
5	Contacto	Comprueba errores, envío válido y limpieza del formulario.	Formulario / estado	Exitosa
6	Navbar	Comprueba contador del carrito, saludo, cierre de sesión, menú móvil y accesos de administrador y vendedor.	Props / evento / estado	Exitosa
7	Home	Comprueba el renderizado del Inicio y la navegación hacia Productos.	Render / navegación	Exitosa
8	QuantitySelector	Comprueba los botones + y -, los callbacks y la cantidad visible.	Props / spies	Exitosa
9	Carrito	Comprueba agregar, aumentar, disminuir, eliminar y contar productos, también a través de CarritoProvider.	Lógica / estado	Exitosa
10	Carrito + localStorage	Comprueba total, lectura, ausencia o error y escritura simulada.	Mock / persistencia	Exitosa

> **Criterio de referencia**
> Estas 10 pruebas son la base que cubre los módulos principales exigidos por la pauta. La explicación y el análisis se centran en ellas; las pruebas complementarias se mencionan en la sección 5 como refuerzo de cobertura.

5. Por qué la suite final tiene 20 pruebas

Durante la etapa final se aplicaron dos formas de optimizar la cobertura. Ambas son válidas y se utilizaron según las necesidades de cada módulo.

Estrategia	Cómo se aplicó	Resultado
Optimizar una prueba ya existente	Se agregaron más escenarios, acciones, expect, mocks o caminos dentro del mismo caso de prueba. Se aplicó en Navbar, carrito y los formularios de Registro, Login y Contacto.	La cantidad de pruebas puede mantenerse, pero cada prueba recorre más lógica.
Agregar pruebas complementarias	Cuando una prueba base no alcanzaba determinadas ramas o funciones, se incorporaron casos adicionales específicos.	Aumenta la cantidad total de pruebas y permite cubrir código que antes no se ejecutaba.

Las 10 pruebas complementarias son un refuerzo de cobertura y no reemplazan a las pruebas base:

Módulo	Pruebas	Qué refuerzan
ProductCard	1	Producto en oferta: muestra el precio de oferta y el original tachado.
productos.js (CRUD)	9	Lista inicial de ocho productos; búsqueda por ID (texto e inexistente); productos bajo el stock crítico; crear (ID nuevo y primer ID); actualizar (solo el indicado e ID inexistente); eliminar (el indicado e ID inexistente).

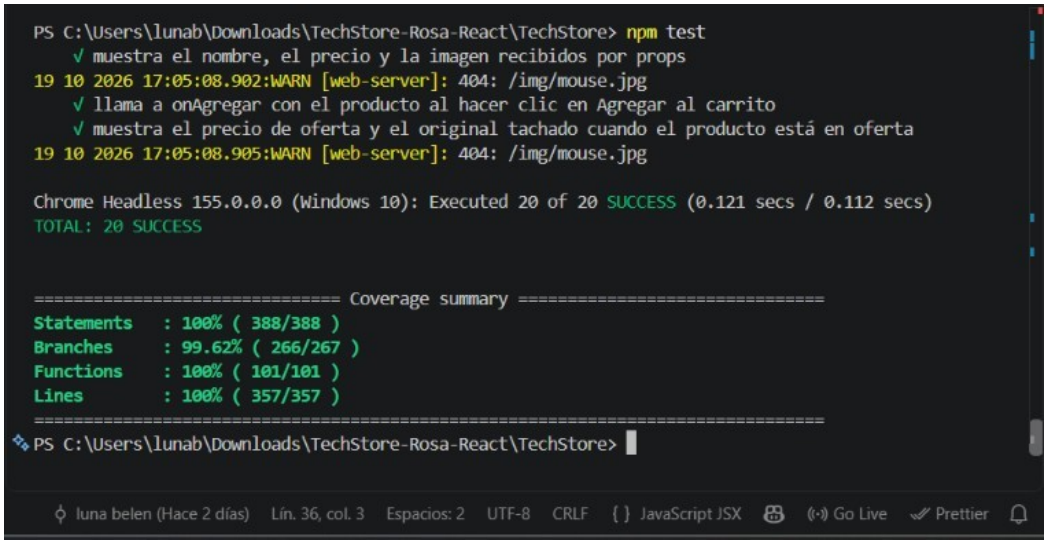
Por esta razón, la base de 10 pruebas se mantuvo como referencia funcional de la pauta, mientras la suite técnica final llegó a 20 pruebas. En las secciones 8 a 11 se indica, módulo por módulo, la evidencia de cada resultado.

> **Resumen de la estrategia**
> La pauta entrega una base de pruebas. Para mejorar la cobertura se usaron dos estrategias: en algunos módulos se ampliaron las pruebas existentes y en otros se agregaron pruebas complementarias que alcanzan ramas y funciones que aún no estaban cubiertas. Por eso la suite final quedó en 20 pruebas.

6. Ejecución final: pruebas y build

6.1 Pruebas con npm test

> **Resultado actual de la rama main**
> 20 pruebas ejecutadas · 20 exitosas · 0 fallidas. Statements 100% · Branches 99,62% · Functions 100% · Lines 100%.



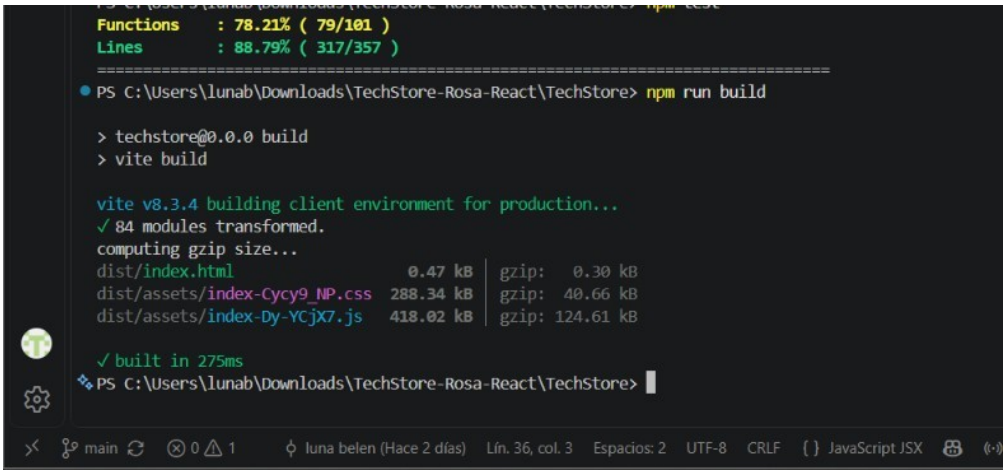
Evidencia 1. Ejecución final de npm test: 20 de 20 SUCCESS y cobertura global actualizada.

La ejecución final confirma que todas las pruebas pasan correctamente. Los avisos 404 visibles corresponden a recursos multimedia solicitados durante las pruebas y no provocan fallos en la suite.

6.2 Verificación del build con npm run build

Además de las pruebas, se comprobó que la aplicación compila para producción con npm run build, que ejecuta vite build. El proceso transformó 84 módulos y terminó sin errores. Las pruebas unitarias no

modifican el código de la aplicación, por lo que el build confirma que la integración final se mantiene compilable.



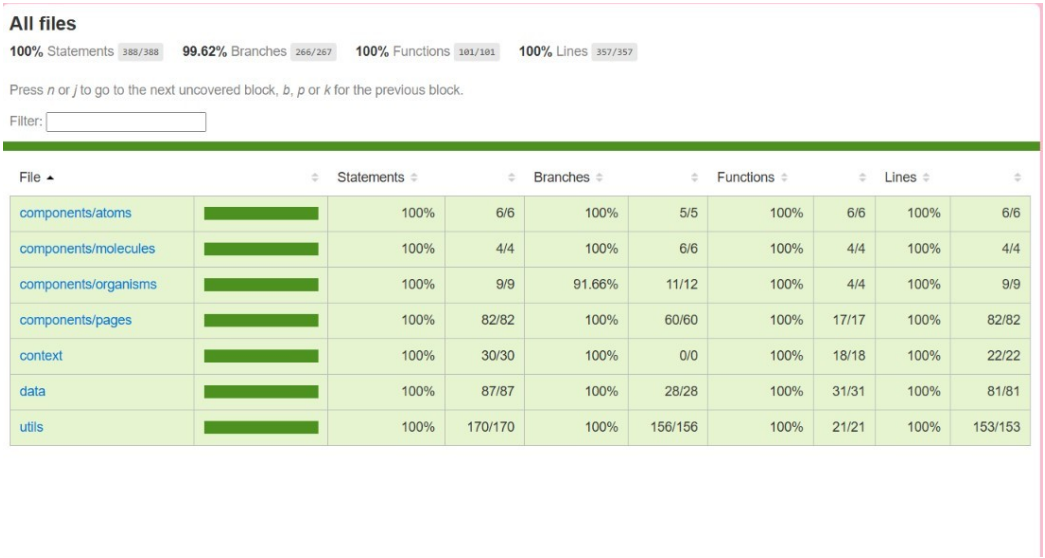
Evidencia 2. Ejecución de npm run build: build de producción completado correctamente.

7. Cobertura global

El resumen global generado en la ejecución final fue:

Statements	Branches	Functions	Lines
100%	99,62%	100%	100%

Estos porcentajes corresponden a los 23 archivos de código que la suite carga e instrumenta durante la ejecución: componentes atómicos, ProductCard, QuantitySelector y FormField, Navbar, las páginas Home, Login, Registro y Contacto, los dos contextos, la capa de datos y las utilidades. No deben interpretarse como cobertura de todos los archivos del repositorio; el alcance exacto se detalla en la sección 12.



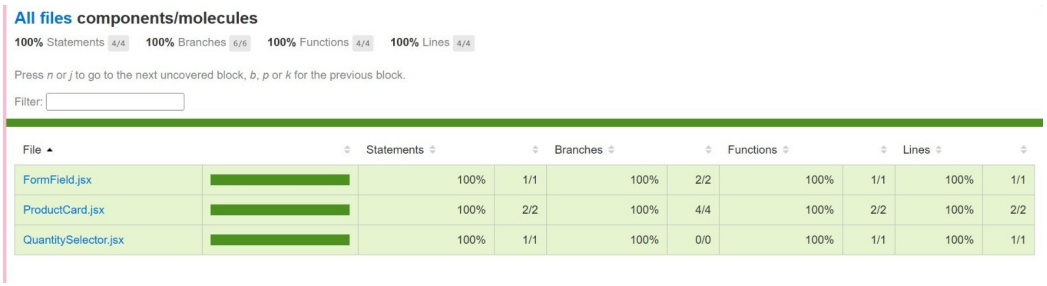
Evidencia 3. Vista All files del reporte Istanbul.

Grupo	Statements	Branches	Functions	Lines
components/atoms	100%	100%	100%	100%
components/molecules	100%	100%	100%	100%
components/organisms	100%	91,66%	100%	100%
components/pages	100%	100%	100%	100%
context	100%	100%	100%	100%
data	100%	100%	100%	100%
utils	100%	100%	100%	100%

La única brecha dentro de los archivos medidos se concentra en components/organisms: Navbar alcanza 11 de 12 ramas, por eso Branches queda en 99,62% global. El resto de las métricas globales llega a 100%.

8. Evidencias de módulos principales

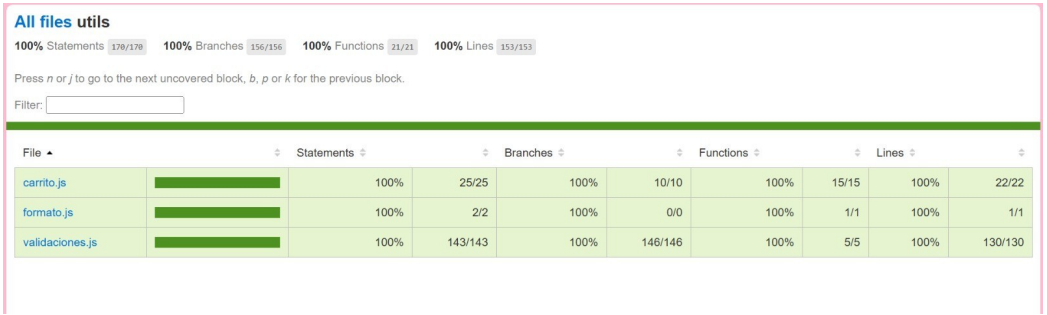
8.1 Components / molecules



Evidencia 4. FormField, ProductCard y QuantitySelector aparecen al 100% en las cuatro métricas.

Esta evidencia respalda pruebas de renderizado, props, callbacks y manejo de cantidad en componentes reutilizables.

8.2 Utils

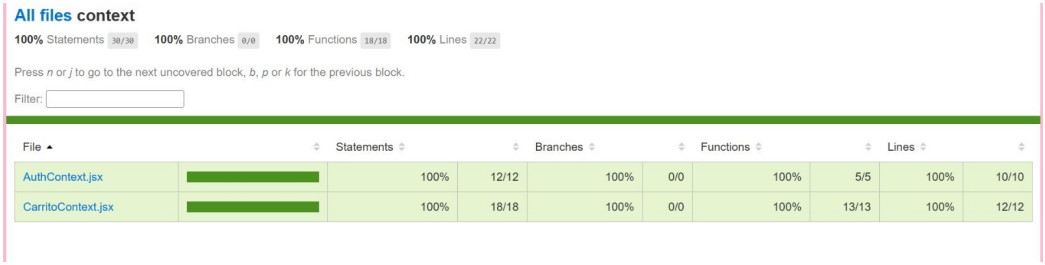


Evidencia 5. carrito.js, formato.js y validaciones.js alcanzan 100% de cobertura.

La lógica pura del carrito, el formateo y las validaciones quedan completamente recorridos por la suite optimizada.

9. Persistencia, datos y contexto

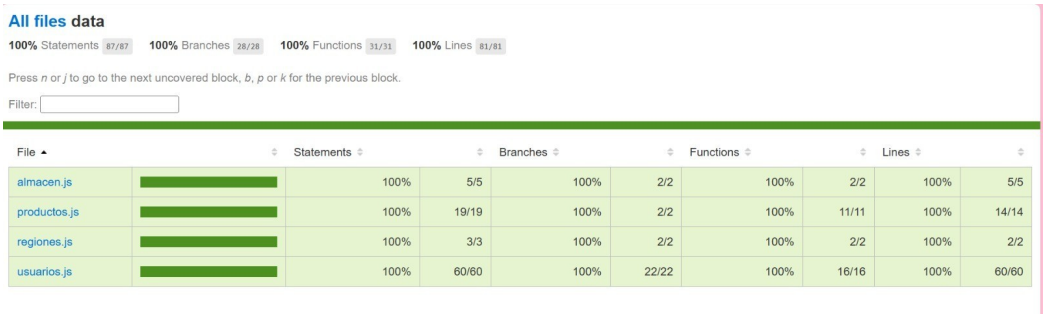
9.1 Context



Evidencia 6. AuthContext.jsx y CarritoContext.jsx alcanzan 100% en Statements, Branches, Functions y Lines.

La optimización final incorporó los escenarios necesarios para recorrer también los proveedores de contexto y no solo las funciones puras asociadas.

9.2 Data

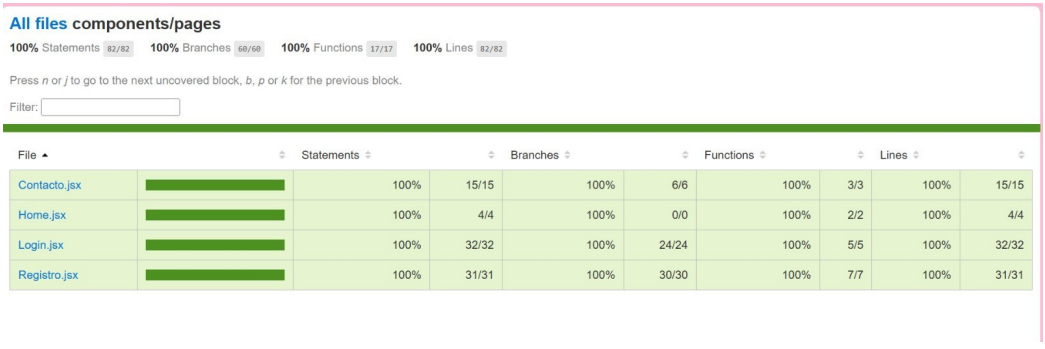


Evidencia 7. almacen.js, productos.js, regiones.js y usuarios.js alcanzan 100%.

Las pruebas complementarias recorren las operaciones utilizadas para simular almacenamiento, catálogo, regiones y usuarios.

10. Páginas y validaciones

10.1 Components / pages



Evidencia 8. Contacto.jsx, Home.jsx, Login.jsx y Registro.jsx alcanzan 100% en las cuatro métricas.

Estas páginas representan los flujos de formularios, autenticación, navegación e inicio cargados por la suite.

10.2 Validaciones

```
                                                    
                                                    
export function validarRegistro(datos) {
  const errores = {}
  const run = datos.run.trim().toUpperCase()
  const correo = datos.correo.trim().toLowerCase()

  if (!datos.nombre.trim()) {
    errores.nombre = 'Debes ingresar tu nombre.'
  } else if (datos.nombre.trim().length > 50) {
    errores.nombre = 'El nombre no puede superar los 50 caracteres.'
  }

  if (!datos.apellido.trim()) {
    errores.apellido = 'Debes ingresar tu apellido.'
  } else if (datos.apellido.trim().length > 100) {
    errores.apellido = 'El apellido no puede superar los 100 caracteres.'
  }

  if (!run) {
    errores.run = 'Debes ingresar tu RUN.'
  } else if (run.length < 7 || run.length > 9) {
    errores.run = 'El RUN debe tener entre 7 y 9 caracteres.'
  } else if (!/^[0-9]{6,8}[0-9K]$/.test(run)) {
    errores.run = 'Ingresa el RUN sin puntos ni guion.'
  } else if (!validarRun(run)) {
    errores.run = 'El RUN ingresado no es válido.'
  }


if (!datos.fechaNacimiento) {
  errores.fechaNacimiento = 'Debes ingresar tu fecha de nacimiento.'
} else {
  const nacimiento = new Date(datos.fechaNacimiento + 'T00:00:00')
  const hoy = new Date()
```

Evidencia 9. Reporte de validaciones con múltiples caminos ejecutados.

La cobertura de validaciones se reforzó con casos válidos y no válidos para recorrer las reglas disponibles y llevar el módulo a cobertura completa.

11. Componentes pequeños y Navbar

11.1 Components / atoms

All files components/atoms

100% Statements 6/6 **100%** Branches 5/5 **100%** Functions 6/6 **100%** Lines 6/6

Press *n* or *j* to go to the next uncovered block, *b*, *p* or *k* for the previous block.

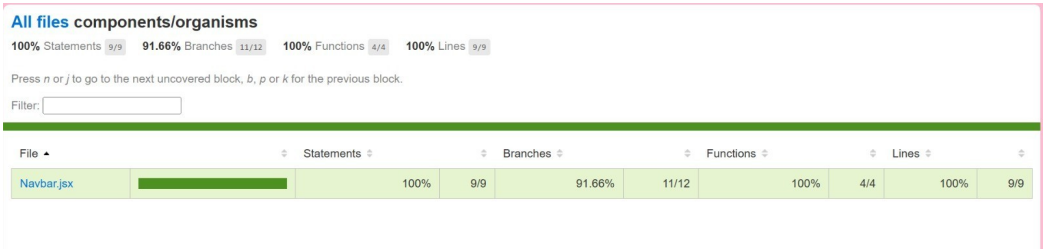
Filter:

File	Statements		Branches		Functions		Lines	
Button.jsx	100%	1/1	100%	1/1	100%	1/1	100%	1/1
Input.jsx	100%	1/1	100%	2/2	100%	1/1	100%	1/1
Label.jsx	100%	1/1	100%	0/0	100%	1/1	100%	1/1
Logo.jsx	100%	1/1	100%	0/0	100%	1/1	100%	1/1
Price.jsx	100%	1/1	100%	2/2	100%	1/1	100%	1/1
ProductImage.jsx	100%	1/1	100%	0/0	100%	1/1	100%	1/1

Evidencia 10. Button, Input, Label, Logo, Price y ProductImage alcanzan 100%.

Los componentes atómicos también quedaron cubiertos como parte de las pruebas complementarias de integración.

11.2 Components / organisms



Evidencia 11. Navbar.jsx: 100% Statements, 91,66% Branches, 100% Functions y 100% Lines.

Navbar mejoró de forma importante. Solo una rama de doce queda pendiente. No se agregó una prueba adicional únicamente para forzar el 100%, ya que la cobertura final de los archivos medidos es prácticamente total y la brecha queda claramente identificada.

12. Análisis final y alcance de la medición

La suite final demuestra dos enfoques de mejora de cobertura. Algunos módulos alcanzaron 100% optimizando las pruebas base: una misma prueba puede recorrer varios escenarios y verificar múltiples comportamientos. En otros módulos fue necesario agregar pruebas complementarias para ejecutar ramas o funciones que no podían alcanzarse de forma suficiente con los casos iniciales.

12.1 Alcance de la medición

El reporte de Istanbul mide únicamente los archivos que la suite importa, por lo que el 100% se refiere a esos 23 archivos. Quedan fuera del reporte, porque ninguna prueba los carga: las vistas del flujo de compra (Carrito, Checkout, CompraExitosa y PagoError), Productos, Categorías, Ofertas, Blog y Nosotros; el panel de administración con sus layouts y la protección de rutas; componentes como CartItem, CartList, CartSummary, Footer, ProductoForm y TablaUsuarios; y los archivos de datos ordenes.js y blogs.js.

Por lo tanto, la cobertura se interpreta como completa sobre el código ejercitado por la suite, y no como una garantía sobre todo el proyecto.

12.2 Mejoras futuras

- Agregar pruebas para el flujo de compra (Checkout, CompraExitosa, PagoError) y el registro de órdenes en ordenes.js.
- Cubrir el panel de administración y la protección de rutas.
- Recorrer la rama pendiente del Navbar (11 de 12) para llevar Branches al 100%.

13. Conclusiones

TechStore cuenta con una suite final de 20 pruebas unitarias ejecutadas correctamente con Jasmine y Karma. Sobre los archivos que la suite carga, la cobertura alcanza 100% en Statements, Functions y Lines, y 99,62% en Branches.

Las 10 pruebas base se mantienen como referencia clara de los principales comportamientos evaluados. Las pruebas restantes corresponden a la etapa de optimización y se documentan por módulo en las secciones 8 a 11 como refuerzo de cobertura.

> **Resultado de cierre**
> 20/20 SUCCESS · Build correcto · Statements 100% · Branches 99,62% · Functions 100% · Lines 100%, sobre los 23 archivos medidos. Dos estrategias de optimización: ampliar pruebas existentes y agregar pruebas complementarias.

14. Referencias

- DSY1104 Evaluación Parcial 2 - pauta y rúbrica del estudiante.
- DSY1104 Evaluación Parcial 2 - Anexo 1 Instrucciones.
- Reporte Istanbul / karma-coverage generado sobre la versión final de main.
- Archivos package.json y karma.conf.cjs del repositorio.
- Repositorio integrado TechStore.